

First practical assignment for Compilers Class

Project Overview

This project implements a basic compiler for a subset of the Kotlin programming language, handling key programming constructs like variable declarations, expressions, conditional statements, loops, and input/output functions. Using a lexical analyzer (`Lexer.x`) and parser (`Parser.y`), the compiler interprets simple Kotlin programs and builds an Abstract Syntax Tree (AST) to represent the program's structure and operations.

Project Components

1. Lexer (`Lexer.x`)

- **Purpose:** The lexer, generated with *Alex*, scans input Kotlin code to produce tokens representing lexical elements.
- **Key Tokens:** Keywords like `val`, `var`, `if`, `else`, `while`, `print`, `readln`; symbols like `+`, `-`, `*`, `/`, `=`, `&&`, `||`, `<`, `>`, `(`, `)`, `{`, `}`, etc.
- **Token Output:** The lexer outputs a sequence of tokens that are passed to the parser for syntactical analysis.

2. Parser (`Parser.y`)

- **Purpose:** The parser, created using *Happy*, analyzes the token stream, verifies the syntactical correctness, and builds the AST.
- **Grammar:**
 - Supports assignments, arithmetic and boolean expressions, conditionals (`if` statements), and loops (`while`).
 - Each construct is associated with AST nodes that represent program statements and expressions.
- **Error Handling:** Custom error messages indicate unexpected tokens or invalid syntax.

3. Abstract Syntax Tree (`AST.hs`)

- **Purpose:** The AST represents the hierarchical structure of the parsed code, allowing further manipulation or evaluation.
- **Main Nodes:**
 - `Program` : Root node representing the main function.
 - `Block` : Represents a block of commands.
 - `Command Types`: Includes variable declarations, assignments, print statements, and control structures like `IfStmt`, `IfElseStmt`, `WhileStmt`.
 - `Expr` : Represents arithmetic and boolean expressions, standalone expressions, and operations within statements.

4. Error Handling

- **Lexical Errors:** The lexer identifies invalid symbols or sequences and stops token generation with an error message.

- **Syntactical Errors:** The parser halts and shows custom error messages if an invalid token sequence is detected, aiding in identifying issues in code structure.

How to Run the Project

1. Compile with Alex and Happy:

```
alex Lexer.x
happy Parser.y
ghc Main.hs Lexer.hs Parser.hs AST.hs
```

2. Run the Compiler:

- Provide `.txt` files containing the Kotlin code inside the `inputs` directory:
- Run the compiled exe

```
./Main
```

3. Output:

- The compiler will parse the Kotlin code, output tokens, and display the AST structure or syntax errors if present.

Challenges and Solutions

- **Ambiguities in Grammar:** Handling both `Expr` and `Block` inside conditional statements required careful distinction in grammar rules to avoid shift/reduce conflicts.
- **Standalone Expressions:** Adjustments were made to allow `Expr` to exist as standalone statements within `Block`s.
- **Error Handling:** Custom messages for unexpected tokens were implemented to aid debugging and provide clearer feedback.

Future Improvements

- **Enhanced AST Operations:** Expanding the AST to support more Kotlin constructs like functions, advanced data types, and complex expressions.
- **Optimization:** Refine grammar rules to further reduce conflicts and enhance parser performance.